Data Structure, Midterm — Solution

A. Single Choice Questions (5% each, 60%)

(1) Algorithm, computable, programming, abstraction: which statement is TRUE?

(C)

- **(A) False.** An algorithm must consist of well-defined, **unambiguous** steps.
- **(B) False.** An algorithm solves **any instance** of the problem that might arise.
- **(C) True.** This combines the definition of *computable* with the requirement that an algorithm terminates in finite time.
- **(D) False.** The definitions are swapped: programming *is* the encoding of an algorithm into a language; abstraction separates the logical and physical perspectives.

Lecture source: deck 01 §1.1–1.4 (cells 6, 8–9, 14, 17–18).

(2) Encapsulation, access specifiers, inheritance: which statement is FALSE?

(A)

- **(A) False, so this is the answer.** A child class inherits the characteristics of its parent and **distinguishes itself by adding** its own data and methods. It may also override inherited methods.
- **(B) True.** This is how `LogicGate` shares its label with the derived gates.
- **(C) True.** C++ has true private data.
- **(D) True.** If no access specifier is given, all members of a `class` are private.

Lecture source: deck 01 §1.12.1 (cells 193–195), §1.13 (cells 228, 231, 239).

(3) Dominant term, growth rate

(D) About 18 seconds

- **(A)** Uses cubic growth (27 times).
- **(B)** Treats quadratic growth as linear (3 times).
- **(C)** Multiplies by the exponent 2 instead of squaring the factor 3 (6 times).
- **(D) Correct.** For large n the dominant term decides: $5(3n)^2/(5n^2) = 9$, so the time grows about 9 times: $2 \times 9 = 18$ seconds.

Lecture source: deck 02 §2.3 (cells 43–47, 51–52).

(4) Benchmark analysis, basic unit of computation, order of magnitude: which statement is TRUE?

(B)

- **(A) False.** Big-O describes growth as n gets large; the n^2 term dominates, so it is $O(n^2)$.

- **(B) True.** This is exactly why the lecture moves from benchmark analysis to counting basic units of computation.
- **(C) False.** The constant is dropped: $T(n) = n + 1$ is $O(n)$.
- **(D) False.** The coefficient of the dominant term becomes insignificant as n grows: $O(n^2)$.

Lecture source: deck 02 §2.1–2.3 (cells 19, 29–31, 33, 43–47, 58).

(5) Logarithmic loops, order of magnitude

(C) 10 iterations; $O(\log n)$

- **(A)** Forgets that $i = 1$ still satisfies $i \geq 1$.
- **(B)** Treats halving as subtracting 2.
- **(C) Correct.** $i = 1000, 500, 250, 125, 62, 31, 15, 7, 3, 1$: ten iterations $= \lfloor \log_2 1000 \rfloor + 1$. Halving gives logarithmic growth (the same count as the number of binary digits produced by Divide by 2).
- **(D)** Treats the loop as counting down by 1.

Lecture source: deck 02 §2.3 (cells 51–52, 96); deck 05 §3.8 (cells 83–84).

(6) Dynamic array insert and erase, capacity: which statement is FALSE?

(A)

- **(A) False, so this is the answer.** The shifting must work **from the end of the array back toward the insertion point**. Copying forward from `idx` overwrites the next value before it has been moved, so data are lost.
- **(B) True.** In the worst case the whole array is shifted.
- **(C) True.** This is the lecture's move count for erase.
- **(D) True.** When it fills up, a bigger raw array (typically twice the size) is allocated and the elements are copied.

Lecture source: deck 03 §A.1 (cells 22–23, 34–41, 45).

(7) Sparse matrix criterion, COO format

(D) It is sparse (NNZ / size $= 0.03$); COO stores $3 \times 1{,}800 = 5{,}400$ numbers.

- **(A)** COO uses three subarrays (row, column, value), not a single position index.
- **(B)** Uses a wrong size of 30,000.
- **(C)** COO also stores the row and column index of every value.
- **(D) Correct.** Size $= 200 \times 300 = 60{,}000$; $1{,}800/60{,}000 = 0.03$. COO keeps three subarrays (row indices, column indices, values), each of length NNZ.

Lecture source: deck 03 §A.4, §A.4.1 (cells 86, 88, 91).

(8) Linked list `add`, external reference `head`: which statement is TRUE?

(B)

- **(A) False.** Nodes can only be reached by following links from `head`; with no reference left, they are lost.
- **(B) True.** This is the lecture's warning about reversing the two lines of `add`.
- **(C) False.** It is the original list that is lost; only the new node remains reachable.
- **(D) False.** No link is changed at the end; from `head` one can only reach the new node.

Lecture source: deck 04 §4.5 (cells 42, 46–47, 49–51).

(9) Ordered linked list, early stop, worst case: which statement is FALSE?

(D)

- **(A) True.** The ordering lets the search stop as soon as possible.
- **(B) True.** `OrderedList::remove` stops early and follows the erase-if-found contract.
- **(C) True.** The new item must be placed where it keeps the list in order.
- **(D) False, so this is the answer.** Stopping early helps only when a larger value is met. A search for a value larger than every item still visits all n nodes, so the worst case remains $O(n)$.

Lecture source: deck 04 §4.6, §4.6.1 (cells 99, 101, 106–109, 116, 122).

(10) Infix-to-postfix conversion: which statement is TRUE?

(A)

- **(A) True.** This is step 3 of the algorithm; `A * B + C` becomes `A B * C +`.
- **(B) False.** The **left** parenthesis is pushed; a right parenthesis pops operators until the left parenthesis is removed.
- **(C) False.** Operands are appended to the output immediately; only operators and left parentheses go on the stack.
- **(D) False.** They are popped, so they come out from the top first (reverse order of pushing).

Lecture source: deck 05 §3.9.2 (cells 127–131, 135–136).

(11) Queue (FIFO), Hot Potato simulation: which statement is TRUE?

(C)

- **(A) False.** The child holding the potato is always at the **front** of the queue.
- **(B) False.** Passing around a circle is FIFO; the eliminated child is the one at the front, not the most recently added one.
- **(C) True.** This is how the simulation models the circle with a queue.
- **(D) False.** It ends when only one name remains (the size of the queue is 1).

Lecture source: deck 05 §3.10, §3.13 (cells 163, 185–187).

(12) Postfix evaluation with an operand stack, operand order

(B) 5

- **(A)** Pops the operands of `-` in the wrong order ($6 - 7 = -1$, then $-1 + 4$).
- **(B) Correct.** $2 \times 3 = 6$; $7 - 6 = 1$; $8/2 = 4$; $1 + 4 = 5$. When an operator is read, the first pop is the **right** operand and the second pop is the left operand.
- **(C)** Pops the operands of `/` in the wrong order ($2/8 = 0.25$, then $1 + 0.25$).
- **(D)** Pops the operands of both `-` and `/` in the wrong order ($-1 + 0.25$).

Lecture source: deck 05 §3.9.3 (cells 143–147, 150).

B. Short-answer questions (8% each, 40%)

(13) Term definitions

Examples are given for reference only; the question does not require them.

(a) **Procedural abstraction**: using a function through its interface without knowing how it is implemented; the caller only knows *what* it does. Example: calling `sqrt` from `<cmath>` without knowing how the square root is computed.

(b) **Worst-case performance**: some algorithms depend on the exact data, not only on the size n. The worst case is a particular data set of size n on which the algorithm performs especially poorly. Example: searching an unordered list for an item that is not there visits all n nodes.

(c) **Base address**: the memory location where an array starts; the element at index i is at `base_address + i * size_of_object`, which makes indexing $O(1)$. Example: an array of 8-byte values starting at 64 has index 4 at $64 + 4 \times 8 = 96$.

(d) **Head**: the external pointer that refers to the first node of a linked list. It is the only entry point: every other node is reached by following `next` links from it. In an empty list, the head refers to no node. Example: in the list `54 26 93`, the head refers to the node holding 54; `add` makes the head refer to the new node.

Lecture source: deck 01 §1.1–1.4 (cells 9–13); deck 02 §2.3 (cells 48–50); deck 03 §A.1 (cells 12–16); deck 04 §4.3 (cells 18–20), §4.5 (cells 37–38, 42).

(14-1) Time complexity analysis (C++)

Answer: $O(n \log n)$

First segment. The inner `while` loop halves j from n down to 1, so it runs $\lfloor \log_2 n \rfloor + 1$ times, **no matter what i is**. The outer loop repeats it n times: $n(\lfloor \log_2 n \rfloor + 1) = O(n \log n)$.

Second segment. $2n$ iterations, $O(n)$.

Overall: counting the updates of `x` as the basic step,
$T(n) = n(\lfloor \log_2 n \rfloor + 1) + 2n = n\lfloor \log_2 n \rfloor + 3n$, so the code is $\boxed{O(n \log n)}$. (Counting other statements as well changes only the constants, not the Big-O.)

Check: for $n = 8$ the first segment runs $8 \times 4 = 32$ times and the second 16 times, 48 updates of `x` in total.

Lecture source: deck 02 §2.3 (cells 33–47, 51–58) and the exercise in §2.4 (cell 96).

(14-2) Time complexity analysis (Python)

Same analysis as (14-1): the Python segment performs exactly the same steps, so the answer is the same.

(15) Infix to postfix conversion: `( A + B ) * C - D / ( E - F )`

(a)

Token	Action	Operator stack (bottom → top)	Output so far
`(`	push	`(`	
`A`	output	`(`	`A`
`+`	push (top is `(`)	`( +`	`A`
`B`	output	`( +`	`A B`
`)`	pop until `(`: output `+`		`A B +`
`*`	push	`*`	`A B +`
`C`	output	`*`	`A B + C`
`-`	pop `*` (higher precedence), push `-`	`-`	`A B + C *`
`D`	output	`-`	`A B + C * D`
`/`	`/` is higher than `-`: push	`- /`	`A B + C * D`
`(`	push	`- / (`	`A B + C * D`
`E`	output	`- / (`	`A B + C * D E`
`-`	push (top is `(`)	`- / ( -`	`A B + C * D E`
`F`	output	`- / ( -`	`A B + C * D E F`
`)`	pop until `(`: output `-`	`- /`	`A B + C * D E F -`
end	pop all: `/`, `-`		`A B + C * D E F - / -`

(b) Postfix: `A B + C * D E F - / -`

(c) Maximum: 4 symbols, `- / ( -`, right after the inner `-` is pushed.

(d) Prefix: `- * + A B C / D - E F`

Fully parenthesized: `(((A + B) * C) - (D / (E - F)))`. Moving each operator to its left parenthesis gives the prefix form.

Lecture source: deck 05 §3.9, §3.9.1, §3.9.2 (cells 105–109, 118–122, 127–131, 135–136).

(16) Ordered linked list trace

(a)

Operation	List	Nodes examined
`add(40)`	`40`	0 (empty list)
`add(15)`	`15 40`	1 (40 > 15 stops)
`add(62)`	`15 40 62`	2 (15, 40; end of list)
`add(27)`	`15 27 40 62`	2 (15, then 40 > 27 stops)
`add(55)`	`15 27 40 55 62`	4 (15, 27, 40, then 62 > 55 stops)

(b) `search(30)` examines 15, 27, 40 (40 > 30 stops): **3 nodes**, returns **false**. `search(70)` examines all **5 nodes**, returns **false**.

(c) The unordered list adds at the head: `55 27 62 15 40`, and `search(70)` examines all **5** nodes. The early stop helps whenever an absent value is smaller than some stored value. A value larger than every item still visits all n nodes, so the worst case of ordered search stays $O(n)$. The price of keeping the order is that `add` costs $O(n)$ instead of $O(1)$.

Lecture source: deck 04 §4.5 (cells 41–46), §4.6 (cells 99, 101, 106–109), §4.6.1 (cells 121–122).

(17) 2D array memory addressing: row-major

(a) Row-major.

Try **column-major** with R rows and element size S: $\text{addr}(Y[i][j]) = b + (jR + i)S$.

$$b + (R + 2)S = 604, \qquad b + 4RS = 532.$$

Subtracting: $(2 - 3R)S = 72$. Since $R \geq 5$ (index 4 is valid), the left side is negative, which is impossible. So the array is **not** column-major.

Row-major with C columns: $\text{addr}(Y[i][j]) = b + (iC + j)S$.

$$b + (2C + 1)S = 604 \quad (1), \qquad b + 4S = 532 \quad (2), \qquad b + (3C + 5)S = 684 \quad (3)$$

$(1) - (2)$: $(2C - 3)S = 72$. $(3) - (1)$: $(C + 4)S = 80$.

Then $2(C + 4)S - (2C - 3)S = 11S = 160 - 72 = 88$, so $S = 8$ and $C + 4 = 10$, $C = 6$.

(b) Element size $S = 8$.

(c) $C = 6$ columns. (The number of rows cannot be determined from these data.)

(d) From (2), the base address is $b = 532 - 4 \times 8 = 500$, so $\text{addr}(Y[4][2]) = 500 + (4 \times 6 + 2) \times 8 = 500 + 208 = \boxed{708}$.

Check: $Y[2][1] = 500 + 13 \times 8 = 604$, $Y[0][4] = 500 + 4 \times 8 = 532$, $Y[3][5] = 500 + 23 \times 8 = 684$.

Lecture source: deck 03 §A.1 (cells 12–16), §A.3.1 (cells 76–83).

Bonus: Deque — Solution

(a) Step 1: `d` = `3` → `8 3` → `1 8 3`. Step 2 removes from the front: `q` = `1` → `1 8` → `1 8 3`. Final `q` = **`1 8 3`**: the queue has been **reversed** (the deque is used like a stack).

(b) Step 1: `d` = `3` → `3 8` → `3 8 1`. Step 2: `q` = `3` → `3 8` → `3 8 1`. Final `q` = **`3 8 1`**, the original order (the deque is used like a queue).

(c) Stack: `addFront` + `removeFront`, or `addRear` + `removeRear` (adding and removing at the same end). Queue: `addRear` + `removeFront`, or `addFront` + `removeRear` (adding at one end, removing at the other).

(d)

Operation	Cost	Reason
`addFront`	$O(1)$	new node links to the old first node; the front pointer moves to it
`addRear`	$O(1)$	the rear pointer gives the last node directly
`removeFront`	$O(1)$	the front pointer moves to the second node
`removeRear`	$O(n)$	the node **before** the last one must become the new rear, and a singly linked list cannot move backward, so it must be found by traversing from the front

Giving every node a link to the previous node as well (a **doubly linked list**) makes `removeRear` $O(1)$.

Lecture source: deck 05 §3.15–3.16 (cells 224–226, 231); deck 04 §A.1 (cells 139–142).